WAR THUNDER · SECOND SCREEN TOOLKIT

WT 战术地图
技术白皮书

把游戏自带的本地数据接口，做成一块平板上的战术地图与飞行仪表盘。
只读、实时、零依赖、无构建。

> “不读取内存，不注入进程，不预测未来——
> 只是把客户端已经算好的数据，画得更好看一点。”

本项目 100% 由 AI 生成
全部代码、脚本与本白皮书由 DeepSeek V4 Flash 模型生成；人类仅提出需求与验证效果。



版本 **1.0**　　许可 **MIT**　　运行环境 **Windows 10/11 + Node.js 18+**　　代码规模 **≈ 45 KB**

01 项目概述

《战争雷霆》客户端运行时会在本机监听一个 HTTP 服务（`127.0.0.1:8111`）， 返回当前战场的结构化数据。本项目把这组数据取出来，实时绘制成平板上的第二屏： 一张战术地图，一套飞行仪表。

战术地图

- 地形底图与 10×10 网格
- 敌我空中单位、地面单位、弹药点
- 战区圆环（被占领时脉冲）
- 击落播报滚动列表

飞行仪表

- 姿态仪：地平线、滚转弧、俯仰梯
- 表速带 / 高度带 / 航向带
- 节流阀、过载、迎角、侧滑、升降率
- 断联与观战状态的明确提示

> **它不做什么**：不读游戏内存、不注入进程、不修改游戏文件；不计算敌我距离或速度；不显示未点亮的敌人；不做任何自动化操作。 所有画面内容都来自小地图与聊天框本来就会显示的信息。

45 KB	0	60 fps	1.2 Mbps
全部源码（两个文件）	npm 依赖	姿态仪刷新	典型带宽占用

为什么需要中间这一层

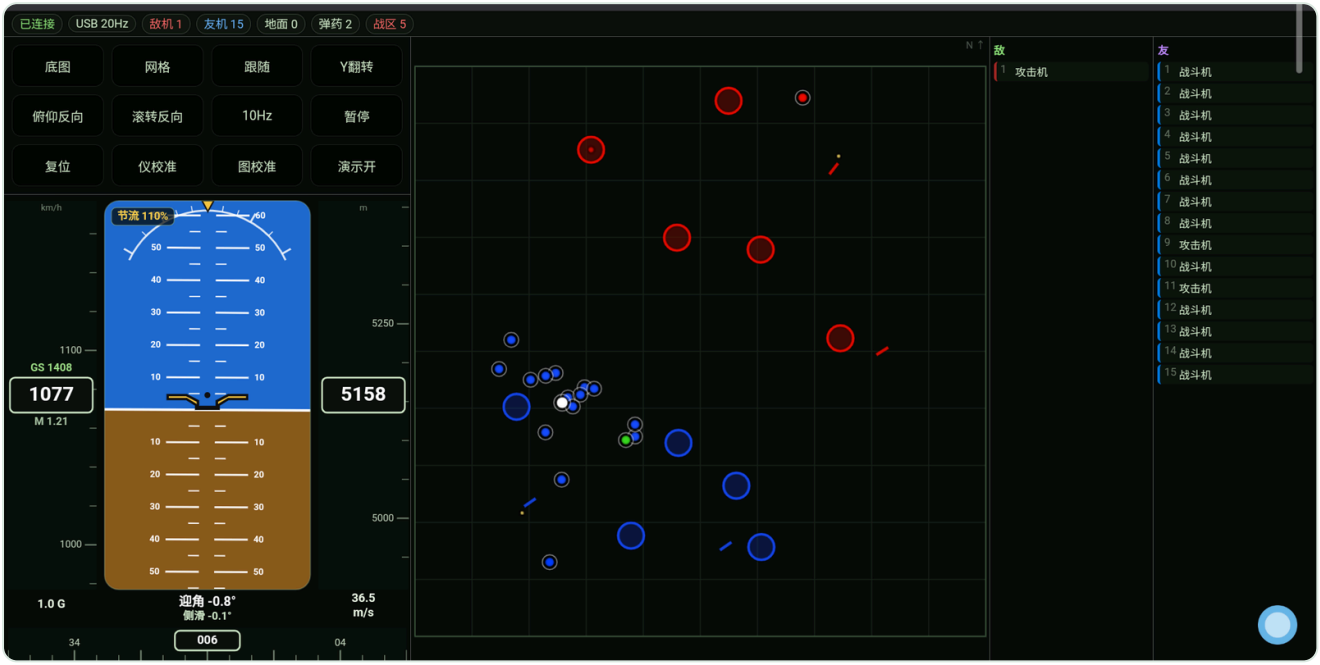
游戏接口只绑定回环地址（`127.0.0.1`），平板在局域网中访问不到。因此在电脑上起一个 35 行的 Node 转发服务，监听 `0.0.0.0:8112`，把请求原样转给游戏，再把页面托管给平板。 这层不做任何业务计算，只做管道。

项目结构

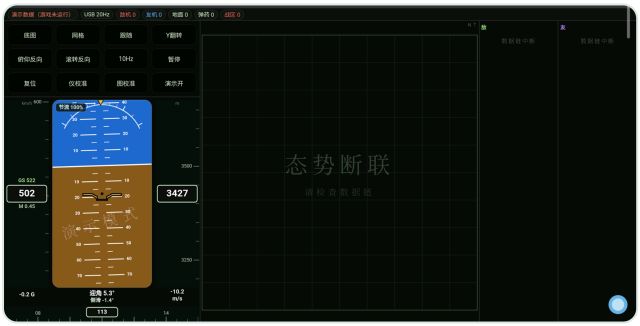
```
WT-Map-OpenSource/
├─ README.md              项目门面：预览、快速开始
├─ LICENSE                MIT
├─ src/
│   ├─ server.js          转发服务（Node 内置模块）
│   └─ index.html         全部前端（地图 + 姿态仪）
├─ scripts/
│   ├─ start.cmd          前台启动
│   ├─ start-hidden.vbs   静默启动
│   ├─ stop.cmd           停止服务
│   ├─ supervisor.vbs     进程守护
│   ├─ watchdog.ps1       守护 supervisor + USB 隧道
│   └─ fix-firewall.cmd   放行 8112
├─ docs/                  技术文档 / 架构说明
└─ screenshots/           效果预览图
```

02 效果预览

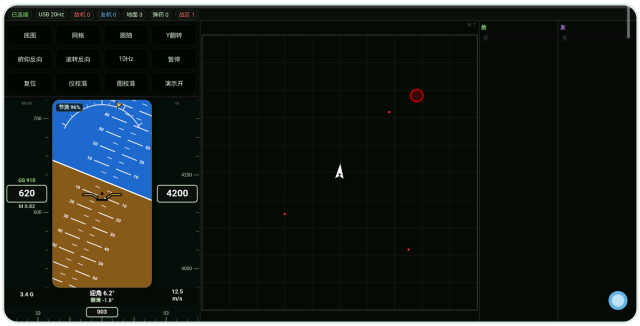
以下截图取自 10.6 英寸、2000×1200 的安卓平板真机；第一张为真实对局数据。



主界面：左侧姿态仪与数据带，中间战术地图，右侧敌我列表



断联：地图中央与敌我栏同时提示



自身箭头：尖端为机头方向



演示模式

演示模式与断联提示

没有任何数据时，界面不会停在最后一帧的“假画面”，而是明确区分三种状态： 对局中无遥测（阵亡/观战）→ 仪表归零；机库/服务未运行 → 地图遮罩； 开启演示 → 合成动画 + 半透明水印。

这样做的目的只有一个：**任何时候都不会把假数据误认为真实数据。**

03 系统架构

三层结构：游戏内置接口 → Node 转发 → 浏览器渲染。每层职责单一，可独立替换。

```
第 0 层  游戏客户端  (aces.exe)
        └ 内置 HTTP 服务  127.0.0.1:8111  只读 · 仅回环
                  |  GET /map_obj.json · /state · /indicators
第 1 层  Node 转发服务  (server.js, 内置模块)
        ├ /api/*       → 反向代理，加 CORS、禁缓存
        ├ 静态文件     → index.html
        └ /api/status → 客户端探测、内网地址枚举
                  |  HTTP
第 2 层  前端  (index.html, 无框架)
        ├ 轮询   poll() 10Hz  ·  instLoop() 10–20Hz
        ├ 状态   haveData · inBattle · linked · demoOn
        ├ 平滑   一阶指数滤波  τ = 35 ms
        └ 渲染   地图 11fps  ·  姿态仪 60fps
```

一次数据往返

时刻	事件
t = 0 ms	`poll()` 发起 `/api/map_obj.json`，上游约 13 KB
t = 20 ms	解析为对象数组，按 `icon` 归类到敌 / 友 / 地面 / 战区
t = 20 ms	`instLoop()` 并行拉取 `/state` 与 `/indicators`
t = 45 ms	判断 `state.valid`：为 `true` 才标记为“有真实数据”
t = 50 ms	地图重绘（约 11 Hz）
每 16.7 ms	姿态仪重绘，读取平滑后的数值

状态机

```
无数据 ──▶ 断联    地图遮罩 + 敌我栏提示
              | map_obj 有单位
              ▼
        对局中 ── state.valid = true ──▶ 已连接（真实数据）
              |                          └ false ─▶ 观战中（仪表归零）
              | 无单位
              ▼
        机库 / 空闲 ── demoOn ? 演示动画 + 水印 : 全零
```

> **为什么状态机要独立于请求：**网络完全断开时请求不会回调，若界面只由回调驱动，就会永远停在最后一帧。 本项目把断联判定放进每帧的渲染循环，服务挂掉的瞬间遮罩就会出现。

04 数据接口

所有数据来自游戏自带的本地服务，无需任何第三方程序。`/state` 的 `valid` 字段是唯一可信判据。

端点	返回内容	本项目用途
`/map_obj.json`	战场对象数组：type、icon、color、blink、x/y（0–1）、dx/dy	地图上的点、线、圆
`/map_info.json`	map_min、map_max、grid_size、map_generation	坐标换算、换图检测
`/map.img`	JPEG 底图，约 200 KB	地图纹理
`/state`	IAS、TAS、H、M、AoA、AoS、Ny、Vy、throttle	仪表数值
`/indicators`	aviahorizon_pitch、aviahorizon_roll、compass	姿态与航向
`/hudmsg`	击杀、受伤事件流	底部播报

关键字段语义

字段	说明
`valid`	**最重要。**为 false 时（机库、阵亡、观战）其余字段全部不可信，前端整体归零
`x` / `y`	归一化坐标 0–1，乘以地图边长得到像素；部分对象用 `sx` / `sy`
`color[]`	多段颜色，用于渐变或闪烁的阵营色
`aviahorizon_pitch`	俯仰角（度）。不同版本符号可能相反，需在界面上校准
`compass`	航向角（度，0–360，北为 0，顺时针为正）
`Ny` / `AoS`	过载（G）与侧滑角（度）

> **为什么不做距离计算：**接口给的是归一化坐标，理论上可换算成米，但那样就跨入了 “推算游戏不会直接显示的信息”的范畴。本项目严格止步于“把已有数据画出来”。

数据可信度判断

场景	valid	处理
正常对局驾驶中	true	正常显示
机库 / 菜单	false	仪表归零，显示演示或断联
对局中阵亡 / 观战	false	仪表归零，状态“观战中”
刚返回机库的数十秒	false	字段残留上一局数值，一律不采信

05 环境与部署

你需要准备什么

类别	最低	推荐
电脑	能运行战争雷霆的任意 Windows PC	任意；本工具单进程 < 80 MB 内存
平板	Android 8.0+，屏幕 ≥ 8 英寸	10–11 英寸，1200p 以上
网络	与电脑同一局域网	5 GHz Wi-Fi，或 USB 数据线
系统	Windows 10 / 11	Windows 11
运行时	**Node.js 18+**（唯一依赖）	Node.js 20 LTS
浏览器	任意现代浏览器	Chrome / Edge 内核
可选	adb（platform-tools）	USB 高速模式与自动化需要

三步启动

#	操作	说明
1	安装 Node.js 18+	`node --version` 能出版本号即可
2	双击 `scripts\start.cmd`	控制台打印本机与平板地址
3	平板打开该地址	建议"添加到主屏幕"，全屏无地址栏

两种链路

方式	平板地址	实际链路	仪表刷新
局域网	`http://<电脑IP>:8112/`	Wi-Fi	10 Hz
USB 隧道	`http://127.0.0.1:8112/`	数据线（adb reverse）	20 Hz

页面读取 `location.hostname` 自动判断，顶栏显示 `USB 20Hz` 或 `WiFi 10Hz`。

USB 模式四步

#	操作
1	平板开启「开发者选项 → USB 调试」
2	数据线连接电脑，平板弹窗点「允许」（可勾选始终允许）
3	电脑执行 `adb devices`，确认设备在线；再执行 `adb reverse tcp:8112 tcp:8112`
4	平板打开 `http://127.0.0.1:8112/`，顶栏应显示 `USB 20Hz`

> **USB 模式的生命周期：**拔线后 `127.0.0.1` 立即失效。建议在平板存两个书签， 插线用隧道地址、不插线用局域网地址。

06 刷新策略与性能

游戏接口由主进程内线程提供，请求过密会影响帧率。本项目把总请求压在 30–50 次/秒，并让绝大多数响应保持在几百字节级别。

轮询频率

接口	USB	Wi-Fi	单次体积
`map_obj`	10 Hz	10 Hz	10–20 KB
`state`	20 Hz	10 Hz	< 1 KB
`indicators`	20 Hz	10 Hz	1–2 KB
`hudmsg`	0.5 Hz	0.5 Hz	10–30 KB
`map_info`	0.125 Hz	0.125 Hz	< 1 KB

合计约 **1.2–1.4 Mbit/s**，约占 260 Mbit/s Wi-Fi 的 0.5%。

为什么要平滑

数据每 50–100 ms 到达一次，而屏幕以 60 fps 刷新。若直接把数值画出来，同一个值会被画 6 帧再突然跳一格， 滚转 300°/s 时每次跳 30°，肉眼非常明显。前端采用一阶指数滤波：

```
k = 1 - exp(-dt / 0.035)        // τ = 35 ms
value += (target - value) * k   // 一阶无过冲
```

代价是约 35 ms 的视觉延迟，换来连续运动；一阶滤波不会过冲，也不会拖影。

为什么要分级渲染

画布	刷新率	理由
姿态仪	60 fps	指针与天地线持续运动，掉帧最明显
地图	约 11 fps	内容准静态，把 CPU 预算让给姿态仪

在入门级平板上，这一条能把掉帧从“可感知”降到“无感”。

守护与自启（可选）

脚本	作用
`start-hidden.vbs`	静默启动服务（无控制台窗口），可放进启动目录
`supervisor.vbs`	每 2 秒检查 node 进程，挂了 3 秒内重启
`watchdog.ps1`	保证 supervisor 存活，并重建 adb 隧道；建议做成每分钟的计划任务

> **为什么用 wscript 而不是 powershell 直接跑：** `powershell.exe` 是控制台程序，计划任务每次触发都会闪一下黑窗； `wscript.exe` 属于 GUI 子系统，可以完全无窗口执行。

07 合规边界

依据《战争雷霆》服务条款：**3.2.8** 禁止未经授权的第三方连接；**3.2.9** 禁止收集正常游戏中不会显示的数据。

行为	判定
显示小地图上已有的敌我单位、战区、占点	安全
显示 TAB / 聊天框可见的击落播报	安全
显示自身遥测（速度、高度、姿态）	安全
显示未点亮的敌人	违规（3.2.9）
计算敌我距离 / 速度、做提前量	违规（3.2.9）
全真等无标记模式下显示敌人	违规
叠加到游戏画面、自动化操作	违规（3.2.3）

> **开源不等于免责。**请在你所在赛区与赛事规则允许的范围内使用，风险自负。

08 已知限制

项	说明
玩家名单	接口不提供花名册，名字只出现在击落播报中
战区血量	对象无 health 字段，只能显示存在 / 消失
具体型号	icon 只给机种，型号仅在播报文本里
地速	无独立字段；游戏无风场，地速恒等于真空速
雷达高度	接口未暴露

09 二次开发

想做什么	改哪里
改端口	`src/server.js` 顶部 `PORT`，或环境变量 `PORT`
加一个仪表	`src/index.html` 的 `drawPFD()`
改地图图标	`drawMap()` 中的 `kindOf()` 分支
换配色	`<style>` 变量与 canvas 的 `fillStyle`

> **本项目 100% 由 AI 生成。**全部代码、脚本与本文档由 DeepSeek V4 Flash 模型生成， 人类仅提供需求描述与效果验证。仓库中不存在人工手写的业务代码。

10 后记

作者：本项目的人类需求提供者 **unknown-obj**。本文初稿由作者手写，DeepSeek V4 Flash 润色。

我为什么想做这个

最开始我打算用 UU 远程这类软件做一块"第三屏"，把游戏画面投到平板上。试过之后发现完全不行： **延迟太大**，机动的时候根本对不上；**画面太糊**，小字看不清；而且**只能看雷达**——可雷达我眼睛往旁边一瞥就能看到。

那看地图呢？问题更荒谬：**我必须先按 M 打开地图界面**，然后才能把这块画面投出去。 为了看一张本来就有的图，得先在游戏里把别的界面挡起来——多少有点脱裤子放屁。 所以只剩官方 App 一条路，但我的谷歌账号被封了，这条路也断了。

那怎么办？我盯着电脑上的 DeepSeek Harness，突然意识到：也许我可以让 AI 来把这件事做出来。

我是怎么提需求的——画框

写代码我不懂，游戏开发、软件开发更是零基础。但我有一支笔，和一块能截图的平板。 我的办法很土，也非常好用：**把截图存下来，用手指在图上画框，写好中文，发过去。** 框一画，歧义就没了——比描述半天"稍微往上一点"有效得多。

往返了二十多轮

这个项目前后**大改了三次**，沟通了**二十多轮**。早期它只是一个能看地图的网页；后来加了遥测面板； 再后来整个推翻重做成"左仪表右地图"的整合布局；最后补上姿态仪、航向带、断联提示、校准面板、USB 高速模式。

每一轮我都挺满意——AI 从来没让我把需求翻译成代码，我只需要说"哪里不对""应该是什么样"。 它会把剩下的活干完，然后我们对着截图继续吵。

一点感想

> **这就是 AI 的伟大之处。**它让一个毫无编程基础、也毫无游戏开发知识的人， 能用可接受的成本、高效地做出自己真正想要的东西。我没有需求文档、没有原型图、没有版本管理， 连"函数"两个字都说不利索——我有的只是"我觉得这里不对"的直觉，和一台能画框的平板。

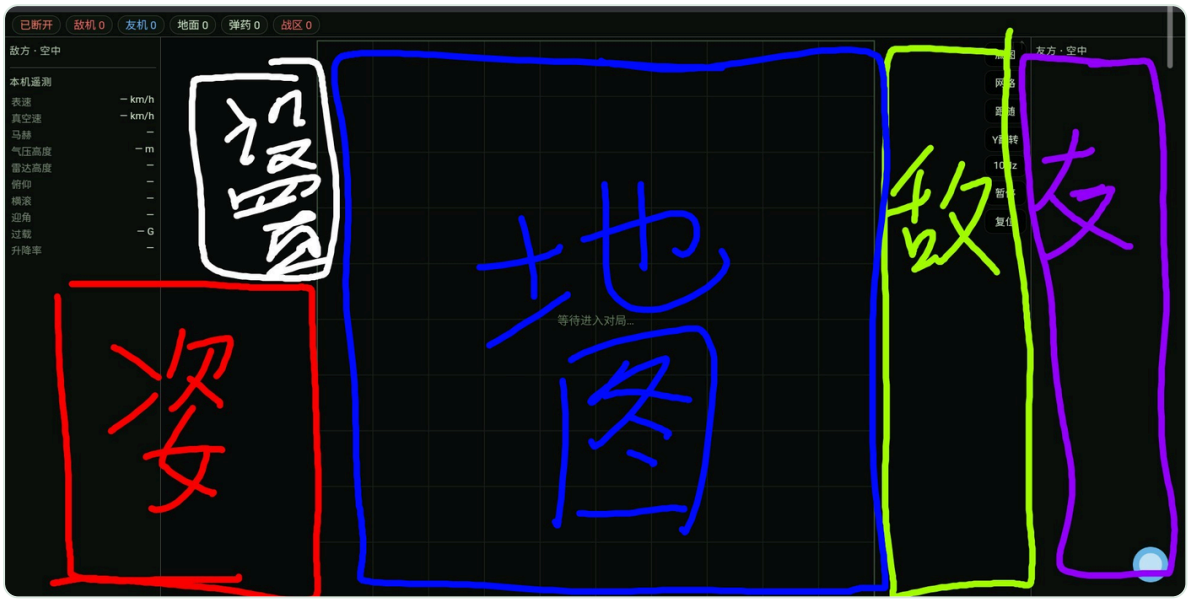
而这些东西，居然真的能变成一个跑在平板上、刷新率 20 Hz、姿态仪会跟着滚转指针走的工具。 那种感觉，大概就是——**第一次看到完整版的时候，震惊瘫坐，仿佛看到核弹爆炸。**

致谢

再次感谢 DeepSeek。也感谢那些愿意被我反复说"这不对"的时刻——没有那些来回，就没有这个项目。 更感谢的是，它让我觉得：**我似乎也拥有了无穷的力量。**

11 需求是怎么长出来的

整个项目的需求表达方式只有一种：在平板上截图，用手指画框，写几个中文字，发过去。



第一版布局：白 = 设置，红 = 姿态仪，蓝 = 地图，绿 = 敌，紫 = 友。左侧面板上还是早期的"本机遥测"列表。



要求把滚转刻度弧"往上挪"：红框画的是**它应该出现的位置**，而不是它现在的位置。

> **为什么画框有效：**自然语言里的"上面一点""大一点""好看一点"都有歧义； 而一个框同时编码了**位置、范围、优先级**三件事，而且不需要任何专业词汇。 对毫无编程背景的人，这是成本最低、信息量最高的需求语言。

结果

两张草图最终对应到第 2 页的界面：左仪表、中地图、右敌我；插线后顶栏显示 `USB 20Hz`。 从划框到成品，中间没有出现过一行需求文档。

附 后记原文（未润色）

以下为作者最初手写的原文，一字未改，仅作对照。文中"她"指代 AI。

后记

后记

我是本项目的"人类需求提供者"

unknown-obj，我本来是准备通过UU远程之类的软件实现"第三屏"，但是使用效果完全不及预期——延迟太大，画面太糊，而且，只能看雷达，但是我明明眼睛往旁边一撇也能看啊。那看地图如何呢？那我必须点m打开界面，这个时候投射到其他设备上，那多少有点脱裤子放屁，而我的谷歌账号被封了，所以我还是用不了官方的APP。

那怎么办呢？我盯上了我电脑上的deepseek harness，也许我可以借助AI的力量来实现我的需求，显而易见的是，她实现了，而且我认为实现的非常好，至于这个东西究竟怎么样，有哪些功能，我想deepseek生成的文档里已经详细的帮我讲述了，我在这里便不再赘述。

在这里我只想说，这就是AI的伟大之处啊，能经济的让一个毫无编程基础的人毫无游戏开发和软件开发相关知识的人也能高效的经济的达到自己的需求，这个项目大改了至少3次，我和deepseek也至少沟通了20次，但是每次我都很满意，我也很享受这种"与AI斗智斗勇的感觉"。也许"震惊瘫坐，仿佛看到核弹爆炸"就是我第一次看到这个项目完整版的心情。

我从来没有想到AI的发展已经这样了——在我打开了平板的允许USB调试之后，她能当着我的面，叫出输入法，飞快的输入IP地址——连上了，我看到了态势图，流畅的姿态仪，也许在战争雷霆的战斗中并不会有太大帮助，然而这很酷，不是吗？我出于好玩的需求，让deepseek做出了这个好玩的产品，这已经是一件非常完美的事情了，但是独乐乐不如众乐乐，我想把这些分享给你们。

再次感谢deepseek，让我似乎也拥有了无限的力量。

我写的很烂，所以——感谢deepseek V4 Flash模型对本文的润色。